

01 · Reverse-engineering primer and glossary (with Thronefall's own evidence)

Answers the question "what is a mapping file, and what else is there?" using the artifacts that now exist for this project. Every count below is measured; the file that proves it is named.

1. The short answer

A **mapping file** is any machine-readable index that answers "where is *X* and what is it?" about a program. Which kinds matter depends on how the game was built:

The game is...	Names in the code are...	The mapping you need	Thronefall
Mono build (DLLs of .NET bytecode)	original (<code>EnemySpawner</code> , <code>BuildSlot</code>)	a code map : classes → members → callers, plus the serialization map that says where data lives in the asset files	This one. <code>Thronefall_Data/Managed/Assembly-CSharp.dll</code> , no <code>GameAssembly.dll</code>
IL2CPP build (native <code>GameAssembly.dll</code>)	stripped, restored from <code>global-metadata.dat</code>	an address/symbol map (IL2CppDumper style)	not applicable
Obfuscated Mono build	scrambled (<code>a</code> , <code>b</code> , <code>c</code>)	a deobfuscation map (old name → new name)	not applicable: names are original
Any build read from outside the process	—	offset tables (memory addresses)	not needed: the bot runs <i>inside</i> the game (BepInEx) and calls methods by name

So for Thronefall, a "mapping file" is **not** a name-restoration file. Decompiling already gave readable code. What was missing was the second half of a Unity game: **the data**. The DLL contains the *rules* (what a wave is, how a spawn works); the numbers (which enemies, how many, at what cost) are stored in Unity asset files (`level15` , `sharedassets1.assets` , `resources.assets`) in a binary format that has **no type information** in this build. Reading them required generating that missing type information from the DLL — which is what `tools/refpack/TfMap` does.

2. Glossary (each term tied to a real artifact)

Term	Meaning	Thronefall evidence
Decompile	bytecode → readable source (ILSpy)	<code>Trainer/decompiled/</code> — 3,056 <code>.cs</code> files of <code>Assembly-CSharp</code>
Disassemble	machine/bytecode → instruction listing	<code>reference/maps/il/Assembly-CSharp/*.il</code> — 5,148 method bodies , 357,204 IL bytes with every call/field/string operand resolved
Mono vs IL2CPP	Unity's two back ends: managed DLLs vs ahead-of-time native code	Mono (no <code>GameAssembly.dll</code> ; <code>MonoBleedingEdge/</code> present). Unity 2022.3.62f2
Metadata token	ECMA-335 id of a type/method/field inside a DLL (<code>0x0600082D</code>)	in every row of <code>reference/maps/code/*.types.jsonl</code> and in the hook map
Serialized field	a value the Unity editor stores in the scene/asset (public or <code>[SerializeField]</code>)	e.g. <code>Spawn.count</code> , <code>Hp.maxHp</code> , <code>BuildSlot.upgrades</code> ; layouts in <code>reference/maps/serialization/layouts.txt</code>
Serialization layout / type tree	the ordered field list Unity needs to read a payload	generated for 1,470 script classes ; asset files here ship <i>without</i> type trees
Byte-exact verification	parse every real object with the layout and require that the parser ends <i>exactly</i> at the object's size	655,330 / 655,330 <code>MonoBehaviours</code> (K: build, 2.13) and 655,157 / 655,157 (Steam build, 2.14) — <code>layout_check.json</code>
Asset typing	knowing the class of each asset object (<code>Transform</code> , <code>MonoBehaviour<Script></code> , <code>Mesh...</code>)	<code>MonoScript</code> table: 4,809 scripts resolve every <code>m_Script</code> pointer; 0 unresolved
Asset ripping / extraction	pulling data out of asset files	<code>tools/refpack/extract_*.py</code> → <code>reference/data/</code>
Hooking (Harmony)	running your code when a game method runs (prefix/postfix)	9 patch points in <code>Trainer/src</code> ; analysed in the hook map
BepInEx	plugin loader for Mono Unity games (Doorstop injects it)	<code>K:\Downloads-IDM\Thronefall\BepInEx\</code>
Inline risk	the JIT may copy a tiny method into its callers, so a patch on it never fires there	flagged per hook (heuristic: ≤ 32 IL bytes, non-virtual)
Runtime dump	reading live objects while the game runs	<code>bot-log.jsonl</code> (65k lines) — used to <i>validate</i> the static extraction
Static extraction	reading everything from files without running the game	all of <code>reference/</code> — no game process was started or touched
Forward model	your own copy of the rules that predicts outcomes without playing	specification in <code>10-forward-model-spec</code> ; the game even ships its own coarse one (<code>EconomySimulator</code> , <code>LevelInfo.virtualBuildings</code>)
Navmesh / A*	precomputed walkable surface + path search (A* Pathfinding Project)	4 baked graphs per scene decoded to triangles: <code>reference/data/navmesh/</code>
Collider / CharacterController	Unity physics shapes; the hero moves with a <code>CharacterController</code>	radius 0.71, height 3.0, step 0.3, slope 45°, skin 0.1 → <code>reference/data/terrain/</code>
Spawn line	polyline of child transforms enemies spawn along	<code>EnemySpawnLine</code> , <code>Spawn.cs:274-297</code> ; coordinates in <code>reference/data/levels/*.json</code>
Layer collision matrix	which physics layers collide	read from <code>PhysicsManager</code> (<code>m_LayerCollisionMatrix</code>)
Drift	what changes between game versions	02-build-fingerprint-and-drift: 2.13 → 2.14
Truth proof	a claim + the measurement that supports or refutes it	12-verification-and-coverage

3. The five maps this project now has

1. **Code map** (reference/maps/code) — every game type and member with tokens, flags, attributes; call graph and field read/write indexes from IL.
Answers: "who calls Hp.TakeDamage?" (32 sites), "what writes EnemySpawner.wavenumber?"
2. **Serialization map** (reference/maps/serialization) — for each script class, the ordered layout of its saved fields. *Answers: "which bytes of Level5 are the wave list?"*
3. **Asset index / data** (reference/data) — decoded values: waves, spawn lines, slots and upgrade trees, enemy/unit/tower stats, UI frames, controls, unlock chain, equippables, balance sheet, localization.
4. **World map** (reference/data/navmesh, terrain, img/maps) — walkable surface, blockers, gates, stand points, routes, chokepoints for all 37 scenes.
5. **Behaviour maps** (reference/docs: state, action, hook) — what the bot can read, what it may do and how, and where it can safely intercept.

4. What deliberately is *not* here

- **Memory addresses/offsets.** Meaningless for an in-process Mono plugin and invalid after every patch.
- **Anything that runs or modifies the game.** All extraction reads files; the game process, the save file and BepInEx/ were never touched.
- **Content not installed:** the Craaghelm/Fangmoor DLC maps are absent from this build (their LevelInfo records exist, their scenes do not).
- **Randomness that only exists at runtime** (Eternal Trials seed, damage variance): documented in the waves doc, not enumerable statically.